

Computer Architecture Lab 2 Report

CSIE B12902075 沈柏宏

Module Explanations

PC

This module takes 4 inputs: reset signal, clock signal, pc write signal, 32-bit next pc. At clock edge, if pc write signal is on, this module forward the next pc value from its input to output, which is connected to instruction memory. Its input, the next pc, is determined by the hazard detection unit and a normal add-4 adder.

Adder

Just a plain 32-bit 2-input adder. It's only used for updating the pc here. Though there is also an adder drawn in hazard detection unit in diagram in spec, I didn't make a instance for it as it's not necessary.

Instruction Memory

This module takes only 1 input: the pc or said the address of instruction. Then it goes to its memory space and outputs the instruction found.

IF ID

This pipeline register forwards pc value and instruction fetched from IF stage to ID stage at clock edge. Now, take stall and flush into consideration, this register actually serves as a source decider for pc and instruction of ID stage:

1. when **reset**: set both pc and instr to 0.
2. when **stall**: keep both pc and instr their old values.
3. when **flush**: set pc to 0 and instr to 32'b10011 to indicate a NoOp.
4. **Otherwise**: propagate pc and instr from IF to ID.

Control

The design of control module is just a extension from lab1. It inputs the opcode, determines the instruction type and outputs the corresponding control signals. The default value for all signals are 0, when NoOp is on or there is no case matching the input opcode.

Registers

A huge module have several inputs and outputs, but its functionality can be summarized in short:

1. read and output register value given its address
2. modify register value at clock edge if RegWrite signal is given.
3. reset all register value to 0 if reset signal is given.

Imm_Gen

This module takes the whole bits of a instruction as input. First it extracts the opcode and funct3. Based on which instruction it is, the module parses the immediate from corresponding bits, and does sign extension.

ID_EX

This module has more than 10 inputs and outputs, but the logic is simple: forward at clock, and zero them at reset.

MUX32_4to1

32-bit 4-input multiplexer. It is used when integrating the source of ALU operand when there is forwarding involved.

MUX32_2to1

32-bit 2-input multiplexer. It is used in a lot of places.

ALU

Completely same design as Lab1. It just does the operation on 2 operand inputs and outputs the result.

ALU_Control

Basically same as Lab1. It uses ALUOp signal to know the type of instruction, determines the alu operation the instruction needs. The 2 types of instructions introduced in this lab (memory and branch) both perform addition at alu.

EX_MEM

Just like ID_EX, propagate value from one stage to another.

Data_Memory

This module maintains the memory space for data.

- **Read:** outputs the data to read when MemRead signal is given.
- **Write:** Write the input data to memory when MemWrite signal is given.

MEM_WB

Same function as ID_EX and EX_MEM. No lengthy description here.

Hazard_Detection

A relatively complex module. It has several inputs from several stages, and outputs some signals and one value for pc which we will discuss below. There are two hazard to detect:

1. **Load Use Hazard:** When current instruction at ID stage has data dependency with its previous instruction which happens to be a load instruction.
2. **Control Hazard** When the current instruction at ID stage is a branch instruction and it actually branches. This also triggers a Flush signal.

Our design to response these two hazards:

1. **Response to Load Use Hazard:** we turn on NoOp signal and Stall signal, turn off PC Write signal to achieve a stall.
2. **Response to Control Hazard:** Use combinational logic to compute target pc for branch first. This pc for branch is connected to a MUX with the normally updated pc value. This MUX is controlled by the Flush signal. Then, the next pc will be updated to pc for branch if there is a flush.

Forwarding

Forwarding unit is the solution to those data hazard that data to use is not written back yet but ready. In order, the forwarding unit checks:

1. Is the data used in EX stage now has an update by the previous non-load-use instruction? If it is, forward the value from MEM stage.
2. Is the data used in EX stage now has an update by the two cycles ago non-load-use instruction? If it is, forward the value from WB stage.

Synthesizability / latch check screenshot

```
• douglas@local:~/ca_lab2/lab2> python3 parse.py
=== Module Areas ===
ALU: 4316.116
ALU_Control: 28.994
Adder: 182.742
CPU: 17351.180
Control: 15.960
Data_Memory: 559.664
EX_MEM: 388.360
Forwarding: 48.412
Hazard_Detection: 286.482
ID_EX: 680.960
IF_ID: 495.558
Imm_Gen: 70.490
MEM_WB: 377.720
MUX32_2to1: 59.584
MUX32_4to1: 164.654
PC: 229.824
Registers: 9434.222
=====
Total area: 34690.922

=== Latch Statistics (PROC_DLATCH pass) ===
No latch inferred count : 21
Latch inferred count    : 0
```

Development environment

- OS: WSL2, Ubuntu 22.04
- Compiler: Yosys 0.9 (git sha1 1979e0b)
- IDE: Visual Studio Code 1.106.2